

CONTENTS

- Overview
- 1 The model is not the agent
- 2 AGENTS.md: how this project works
- 3 Nesting and precedence
- 4 Agent skills: how to do a kind of task
- 5 MCP: reaching tools and data
- 6 A2A: agents talking to each other
- 7 Subagents: work that will not fit one context
- 8 The five in one picture
- Glossary
- Check yourself
- Where to go next

Five Building Blocks of a Modern AI Agent

AGENTS.md, skills, MCP, A2A and subagents — what each one is for, and why they are not interchangeable.

For engineers building agents who need to know which mechanism solves which problem · 26 July 2026

Every section, key point, term and question from the full lesson is here. The extended explanations and the additional examples are not.

[Watch the source video](#)

[Read the full lesson](#)

[Read the highlights](#)

BEFORE YOU START

- What a context window is and that it is finite
- Familiarity with an agent calling tools
- Comfort reading Python and JSON

Overview

A capable agent is not a capable model. The model generates text and reasons; on its own it is a conversational partner. Everything that makes it an agent is scaffolding built around it, and five pieces cover most of what a frontier coding agent actually runs on.

They divide cleanly by the problem each solves. Two shape behaviour from the inside: AGENTS.md says how this particular project works, and skills say how to perform a particular kind of task. Two reach outward: MCP connects the agent to tools and data, and A2A connects it to other agents. The fifth, subagents, handles work that will not fit in a single context window.

That division is the useful part, because the five are routinely confused. Knowledge that should be a skill ends up permanently loaded in AGENTS.md. Coordination that wants A2A gets built as an MCP tool. Work that needs a subagent gets crammed into one context until quality collapses. Each mistake has a recognisable symptom, and knowing which piece was designed for which problem is most of the fix.

KEY TAKEAWAYS

- ✓ The model is not the agent — the instruction layer wrapped around it is what makes it one.
- ✓ AGENTS.md is a project-level instruction file, read whenever the agent starts work there.
- ✓ Nested AGENTS.md files stack, with those closer to the working directory taking precedence.
- ✓ A skill is a folder with a SKILL.md whose description tells the agent when to invoke it, so it stays out of context until needed.
- ✓ MCP is a standard interface between an agent and external tools or data, replacing per-target connectors.
- ✓ A2A is a standard for agents to discover and delegate to each other, using a published agent card.
- ✓ MCP is asymmetric — agent to tool. A2A is between peers, each autonomous.
- ✓ Subagents run in their own fresh context and return a result, keeping the parent's context clean and enabling parallel work.

01 The model is not the agent

0:00

A model generates text and reasons. What turns it into an agent is the instruction layer wrapped around it.

KEY POINTS

- The model supplies generation and reasoning, nothing else.
- A model alone is a conversational partner, not an agent.
- The instruction layer around the model is what makes it agentic.
- A better model improves each step; it does not supply project knowledge or access.
- The five pieces split into: shape behaviour, reach outward, and scale beyond one context.

The five, by the problem each solves

Worth memorising in this shape rather than as a list. Nearly every design mistake in this area is reaching for the wrong row.

inside the agent – shapes behaviour

AGENTS.md	how this project works	always loaded
skills	how to do a kind of task	loaded on match

reaching outward

MCP	agent → tools and data	client / server
A2A	agent ↔ agent	peer to peer

beyond one context

subagents	spawn a child with its own context, get a result back	
-----------	---	--

02 AGENTS.md: how this project works

0:40

A Markdown file at the project root that the agent reads whenever it starts work there.

KEY POINTS

- A Markdown file at the project root, read whenever the agent starts work there.
- Holds commands, conventions, setup steps and formatting rules — a README for agents.
- Instructions are executed when contextually relevant, not merely displayed.
- Introduced by OpenAI, later contributed to the Agentic AI Foundation under the Linux Foundation.
- Some agents use another filename for the same idea, such as CLAUDE.md.
- It is loaded every session, so only broadly relevant content belongs in it.

What earns its place in the file

Note how much of it is prohibitions. The repeated mistakes cost more than the missing facts, and 'do not touch this' is the part nobody thinks to write down until an agent has already touched it.

```
# AGENTS.md

## Commands
test:  pnpm test          run before every commit
build: pnpm build         never call tsc directly
lint:  pnpm lint --fix

## Conventions
- Conventional Commits for messages: feat|fix|chore(scope): subject
- PR titles match the commit subject.
- Money is integer cents. Never float.

## Do not
- Do not edit src/generated/ - it is code-generated.
- Do not add dependencies without asking.
- Do not modify migrations that have already shipped.
```

03 Nesting and precedence

2:02

Several AGENTS.md files can coexist. The one closest to the working directory wins.

KEY POINTS

- Multiple AGENTS.md files can exist in one repository.
- Files closer to the working directory override those further up.
- Root holds what is universal; sub-projects hold what is local.
- It behaves like scope resolution: nearest statement about a topic wins.
- Duplicating a root rule in a child creates a copy that drifts and then silently wins.
- There is no way to lock a rule against being overridden further down.

A monorepo with three scopes

Working in `services/api/`, the agent applies the root then the api file. The test command is the local one; the commit convention is the root one, since nothing nearer contradicts it.

```
repo/
├── AGENTS.md           conventional commits; never edit generated/
├── services/
│   └── api/
│       └── AGENTS.md   test: pytest -q ; python 3.12 ; ruff
└── web/
    └── AGENTS.md       test: pnpm test ; node 22 ; prettier

# working in services/api/ → root, then api
# test command   : pytest -q           (from api)
# commit format  : conventional       (from root, uncontested)
```

04 Agent skills: how to do a kind of task

2:44

A folder with a SKILL.md whose description tells the agent when to pull it in — and which stays out of context until then.

KEY POINTS

- A skill is a folder containing SKILL.md plus any scripts and resources.
- The description tells the agent when to invoke the skill.
- Unmatched skills consume no context at all.
- Skills are an open standard supported by multiple platforms.
- AGENTS.md is bound to a project; a skill is bound to an activity and travels.
- The description is a routing key, not documentation — a vague one makes the skill unreachable.

A skill folder

Only the frontmatter is visible until the skill matches. The supporting files are not read until execution actually reaches the step that needs them.

```
skills/slide-deck/
├─ SKILL.md           when to use it + the steps
├─ template.pptx      opened during execution
└─ build_deck.py      run at the final step

# SKILL.md
---
name: slide-deck
description: Build a PowerPoint deck. Use when the user asks for
  slides, a deck, a presentation, or a readout.
---

1. Confirm the audience and the time slot. Ask if not given.
2. Draft an outline, one line per slide. Get it approved.
3. Fill template.pptx, one idea per slide, no paragraphs.
4. Run build_deck.py and report the output path.
```

05 MCP: reaching tools and data

4:49

An open protocol that wraps any tool or data source in a standard interface, so one connector serves every agent.

KEY POINTS

- Agents must reach APIs, databases, developer tools and SaaS platforms, each with its own interface.
- MCP is an open protocol; an MCP server wraps a target in a standard interface.
- Any MCP-speaking agent can use any MCP server.
- Started at Anthropic, now governed under the Agentic AI Foundation at the Linux Foundation.
- Integration effort goes from agents × systems to agents + systems.
- Tools are actions, resources are readable data — modelling everything as a tool is a common error.
- The server is where scope and permission should be enforced.

Why a protocol instead of connectors

Each line on the left is bespoke code somebody maintains. On the right, each server is written once and every current and future agent can use it.

without a standard	with MCP
agent A → Notion	agent A ┘
agent A → Stripe	agent B ┘ ┌→ MCP → Notion
agent A → Postgres	agent C ┘ ┌→ Stripe
agent B → Notion	→ Postgres
agent B → Stripe	
...	
N x M connectors	N + M

06 A2A: agents talking to each other

6:17

An open protocol for agent-to-agent communication, where each agent publishes a card describing what it does and how to reach it.

KEY POINTS

- A2A is an open protocol for communication between agents.
- Each agent publishes an agent card describing what it does and how to reach it.
- Other agents read the card to work out how to delegate.
- It comes from Google and is now an open standard under the Linux Foundation.
- MCP is asymmetric client-to-server; A2A is between autonomous peers.
- An MCP call returns; an A2A delegation is a long-running task needing identity and progress.
- The receiving agent decides how to accomplish the request, not which function to run.

An agent card

Illustrative shape rather than an exact schema — the point is what a card has to carry. Discovery needs a stable location, delegation needs an endpoint and auth, and routing needs a description specific enough to choose on.

```
{
  "name": "finance-approvals",
  "description": "Approves vendor spend against policy and budget.",
  "skills": [
    {"id": "approve_contract",
     "description": "Review a vendor contract and approve or reject spend."},
    {"id": "check_budget",
     "description": "Report remaining budget for a cost centre."}
  ],
  "endpoint": "https://finance.internal/a2a",
  "auth": {"type": "oauth2", "scopes": ["spend:approve"]}
}
```

published at a well-known location so other agents can find it

07 Subagents: work that will not fit one context

8:24

A child agent spawned with its own fresh context, which does a piece of work and returns a result.

KEY POINTS

- Two motivations: work too large for one context, and work that is embarrassingly parallel.
- A subagent runs in its own fresh context, does a job and returns a result.
- The parent's context stays clean because it receives only the result.
- Not a formal standard, but near-identical across modern agent systems.
- Whatever the child leaves out of its summary is lost to the parent.
- Cost scales with the number of children; you are buying latency with tokens.
- Do not fan out when subtasks need each other's findings.

Fan out, then aggregate

The parent never sees the files. It sees twenty short reports, which is what keeps its context usable. Note that a failing child returns a value rather than raising — one bad check should not lose the other nineteen.

```
def review(functions, parent_ctx):
    reports = run_parallel([
        spawn_subagent(
            task=f"Review {fn.path}:{fn.name} for error handling only. "
                f"Reply with at most 5 lines: findings or 'none'.",
            context="fresh",
        )
        for fn in functions
    ], on_error=lambda fn, e: f"{fn.name}: FAILED ({e})")

    return summarise(reports)      # parent sees ~20 short reports,
                                   # never the 20 files
```

08 The five in one picture

10:28

Two shape behaviour, two reach outward, one scales past a single context — and the symptoms tell you which is missing.

KEY POINTS

- AGENTS.md and skills shape behaviour from inside the agent.
- MCP reaches tools and data; A2A reaches other agents.
- Subagents handle work that will not fit one context window.
- Procedural knowledge in AGENTS.md shows up as a context budget spent before work begins.
- Coordination built as an MCP tool breaks when the far side needs to ask something.
- Diagnose the missing piece before questioning the model.

Symptom to missing piece

Each symptom points at exactly one of the five, and the fixes are not interchangeable. Note that none of the rows is answered by a better model.

symptom	what is missing
"it uses the wrong test command every time"	AGENTS.md
"it knows the facts, botches the procedure"	a skill
"it cannot reach our database at all"	an MCP server
"two agents duplicate each other's work"	A2A
"quality drops halfway through long tasks"	subagents
"the context is full before it starts"	a skill (content is in AGENTS.md)

Glossary

AGENTS.md

A Markdown file at a project root holding commands, conventions and prohibitions, read whenever an agent starts work in that project.

Agentic AI Foundation

The body under the Linux Foundation that now governs several of these standards, including AGENTS.md and MCP.

Nesting / precedence

Several AGENTS.md files applied from the root down, with the file closest to the working directory overriding those above it.

Agent skill

A folder containing a SKILL.md plus scripts and resources, describing how to perform one kind of task.

SKILL.md

The file defining a skill: metadata including the description that tells the agent when to invoke it, and the steps to follow.

Skill description

The one visible part of a skill before loading. It functions as a routing key, so a vague description makes the skill unreachable.

MCP

Model Context Protocol: an open protocol connecting AI applications to tools, data sources and workflows through a standard interface.

MCP server

The component wrapping a tool or data source so any MCP-speaking agent can use it, and the right place to enforce permission.

A2A

Agent to agent: an open protocol letting autonomous agents discover one another and delegate work.

Agent card

A published description of what an agent does and how to talk to it, which other agents read in order to delegate.

Subagent

A child agent spawned by a parent, running in its own fresh context, doing one piece of work and returning a result.

Context isolation

The property that makes subagents useful: the child holds the bulk of the material and the parent receives only the conclusion.

Embarrassingly parallel

Work that splits into independent pieces needing no coordination — the case where fanning out to subagents pays.

Check yourself

Answer first, then open each one.

A guide for generating quarterly reports is added to AGENTS.md and the agent starts running out of context on unrelated tasks. What went wrong?

AGENTS.md is loaded on every session regardless of the task, so occasional, activity-specific knowledge placed there is paid for constantly. It belongs in a skill, which stays out of context — costing only its description — until a request matches it.

A monorepo root AGENTS.md says to use conventional commits, and a sub-project file repeats that rule with slightly different wording. Why is this worse than not repeating it?

Files closer to the working directory override those above, so the duplicated copy is the one that applies under that sub-project. It will drift from the root over time and silently win, producing behaviour that contradicts the rule everyone believes is in force — and there is no way for the root to lock a value against being overridden.

A skill has excellent instructions and is never invoked. Where is the fault almost certainly?

In the description. It is the only part visible before loading, so it acts as the routing key. If it does not name the situation and the words a user would actually say, nothing matches it and the instructions are never read — the decision to load happens before they are visible.

Why does MCP change integration effort from a product into a sum?

Without a standard, each agent-to-system pairing needs its own connector, so the work scales as agents times systems. With MCP each system needs one server and each agent speaks the protocol once, so it scales as agents plus systems — and any new agent immediately gets every existing server.

What is the essential difference between calling an MCP tool and delegating over A2A?

An MCP call is asymmetric and specifies which function to run; the far side executes and returns. An A2A delegation goes to an autonomous peer that decides how to achieve the outcome, may take a long time, may ask a clarifying question, and may decline. Treating delegation as a function call breaks the first time any of those happens.

You spawn twenty subagents to review twenty files and get back twenty vague paragraphs. Why is this unrecoverable without redoing the work?

Each child's context is discarded when it finishes, so anything not in its reply is gone — the parent never saw the files and cannot query what the child read. The instruction given to a child therefore determines what survives, which is why it should specify the exact shape of the reply rather than asking for a general account.

Where to go next

- Audit your own AGENTS.md and move anything task-specific into a skill, then measure how much of the context budget you get back on an unrelated task.
- Set up a nested AGENTS.md in a monorepo and deliberately create a conflicting rule, to see for yourself which one the agent follows.
- Rewrite the descriptions of skills that never trigger as explicit trigger conditions naming user phrasings, then count how often they are selected afterwards.
- Write an MCP server for one internal system and notice how much of the work is deciding what should be a tool versus a resource rather than writing the integration itself.
- Model one hand-off between two of your services as an A2A delegation and enumerate the states you have to handle — completed, rejected, input-required — that a function call never has.
- Take a long single-agent task where quality degrades and split the bulk reading into a subagent, then compare the parent's context usage before and after.
- The MCP entry here has a fuller treatment elsewhere in this collection, and the memory lesson covers how AGENTS.md and skills map onto semantic and procedural memory — this page is the map, those are the depth.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.